

second set. And here is a demonstration of these concepts:

```
$ maxima -q
(%i1) union({1, 2}, {1, 3, 4}, {1, 2, 6, 7});
(%o1) {1, 2, 3, 4, 6, 7}
(%i2) intersection({1, 2}, {1, 3, 4}, {1, 2, 6, 7});
(%o2) {1}
(%i3) setdifference({1, 2}, {1, 3, 4});
(%o3) {2}
(%i4) quit();
```

Other basic set operations provided by Maxima are:

- **cardinality()** - returns the number of distinct items in a set
- **elementp()** - checks for an item to be a member of a set
- **empty()** - checks for the emptiness of a set
- **setequalp()** - compares two sets for equality
- **disjointp()** - checks for no common items in two sets
- **subsetp()** - checks for the first set to be a subset of the second set

The following walk-through demonstrates all of these operations:

```
$ maxima -q
(%i1) S1: {}$
(%i2) S2: {1, 2, 3}$
(%i3) S3: {3, 1, 5-3}$ /* Same as S2 */
(%i4) S4: {a, b, c}$
(%i5) S5: {2, 1, 2}$
(%i6) cardinality(S1);
(%o6) 0
(%i7) cardinality(S2);
(%o7) 3
(%i8) cardinality(S3);
(%o8) 3
(%i9) cardinality(S4);
(%o9) 3
(%i10) cardinality(S5);
(%o10) 2
(%i11) elementp(b, S3);
(%o11) false
(%i12) elementp(b, S4);
(%o12) true
(%i13) empty(S1);
(%o13) true
(%i14) empty(S2);
(%o14) false
(%i15) setequalp(S1, S2);
(%o15) false
(%i16) setequalp(S2, S3);
(%o16) true
(%i17) disjointp(S1, S2);
(%o17) true
(%i18) disjointp(S2, S3);
(%o18) false
```

```
(%i19) disjointp(S3, S4);
(%o19) true
(%i20) disjointp(S3, S5);
(%o20) false
(%i21) subsetp(S1, S2);
(%o21) true
(%i22) subsetp(S2, S3);
(%o22) true
(%i23) subsetp(S3, S2);
(%o23) true
(%i24) subsetp(S3, S4);
(%o24) false
(%i25) subsetp(S5, S3);
(%o25) true
(%i26) subsetp(S3, S5);
(%o26) false
(%i27) quit();
```

Playing with set elements

After clearing the fundamentals, mostly through numerical examples, it is now time to have some fun with symbol substitution of Maxima. So let's play around some more:

```
$ maxima -q
(%i1) S: {a, b, c, a};
(%o1) {a, b, c}
(%i2) S: {a+b, b+c, c+d, d+a};
(%o2) {b + a, c + b, d + a, d + c}
(%i3) subst(a=c, S);
(%o3) {c + b, d + c}
(%i4) subst([a=c, b=d], S);
(%o4) {d + c}
(%i5) subst([a=c, b=d, c=-d], S);
(%o5) {}
(%i6) subst([a=1, b=2, c=-3], S);
(%o6) {- 1, 3, d - 3, d + 1}
(%i7) T: {S, {S}};
(%o7) {{b + a, c + b, d + a, d + c}, {{b + a, c + b, d + a, d + c}}}
(%i8) subst([a=c, b=d, c=-d], T);
(%o8) {{}, {{}}}
(%i9) subst([a=1, b=2, c=-3], T);
(%o9) {{- 1, 3, d - 3, d + 1}, {{- 1, 3, d - 3, d + 1}}}
(%i10) quit();
```



By: Anil Kumar Pugalia

The author is a hobbyist in open source hardware and software, with a passion for mathematics. A gold medallist from NIT Warangal and IISc Bangalore, mathematics and knowledge-sharing are two of his many passions. Apart from that, he shares his experiments with Linux and embedded systems through his weekend workshops. Learn more about him and his experiments at <http://sysplay.in>. He can be reached at [email@sanka-pugs.com](mailto:sanka-pugs.com).